



ASCI

# Atribut dan Method



## Atribut

**Atribut** adalah data yang melekat pada suatu objek merupakan semacam informasi yang menjelaskan karakteristik atau kondisi objek tersebut. Namun, tidak semua atribut berperilaku sama. Di Python, atribut terbagi menjadi dua jenis yang berbeda secara mendasar dalam hal kepemilikan, tempat penyimpanan, dan cara kerjanya yaitu **atribut instance** dan **atribut kelas**. Memahami perbedaan keduanya penting, sebab kesalahan dalam menentukan jenis atribut dapat menimbulkan *error* pada program, terutama ketika program melibatkan banyak objek sekaligus.

| Atribut Instance                             | Atribut Kelas                               |
|----------------------------------------------|---------------------------------------------|
| Dimiliki oleh setiap objek secara individu   | Dimiliki oleh kelas (dibagi ke semua objek) |
| Didefinisikan di dalam <code>__init__</code> | Didefinisikan dalam badan                   |
| Berbeda tiap objek                           | Sama tiap objek (kecuali di override)       |
| Diakses menggunakan <code>self</code>        | Diakses menggunakan nama                    |
| Digunakan ketika data yang unik tiap entitas | Digunakan pada data yang dipakai berulang   |

## Atribut Instance

**Atribut instance** adalah atribut yang nilainya **unik** untuk setiap objek yang dibuat dari suatu kelas. Dimana meskipun dua objek berasal dari kelas yang sama, nilai atribut instance nya bisa sama sekali berbeda satu sama lain dikarenakan masing-masing objek menyimpan salinan nilainya sendiri. Atribut jenis ini didefinisikan di dalam method `__init__` menggunakan `self.nama_atribut`, sehingga nilainya ditentukan setiap kali sebuah objek baru dibuat. Karena sifatnya yang personal bagi tiap objek, atribut instance merupakan jenis atribut yang paling sering digunakan karena hampir seluruh data yang mendeskripsikan identitas suatu objek (nama, umur, saldo, status, dan sejenisnya) biasanya disimpan sebagai atribut instance.



Contoh kode:

```
class Tim:
    def __init__(self, nama, ceo):
        self.nama = nama
        self.ceo = ceo

rrq = Tim("RRQ Hoshi", "Pak AP")
evos = Tim("Evos Legends", "Hartman Harris")

print(rrq.nama)      # RRQ Hoshi
print(evos.nama)     # Evos Legends
```

Cara mengubah atribut instance dapat melalui langsung lewat objeknya dan perubahan itu hanya berlaku untuk objek tersebut

```
rrq.ceo = "Pak AP Manullang"

print(rrq.ceo)      # Pak AP Manullang
print(evos.ceo)     # Hartman Harris
```

## Atribut Kelas

**Atribut kelas** adalah atribut yang dimiliki oleh sebuah . Atribut ini didefinisikan langsung di badan kelas, di luar method apapun sehingga nilainya sudah ada sebelum objek dibuat dari kelas tersebut. Karena nilainya cuma disimpan satu kali di , semua objek yang dibuat dari itu akan melihat nilai yang sama persis. Bayangkan seperti papan pengumuman bersama dimana semua orang membaca papan yang sama, bukan punya catatan masing-masing. Atribut kelas biasanya dipakai untuk data yang memang seharusnya sama untuk semua objek, misalnya nama instansi atau nilai default yang berlaku umum. Contohnya dapat dilihat dari kode berikut:



```
class Tim:
    nama_liga = "MPL Indonesia"

    def __init__(self, nama, ceo):
        self.nama = nama
        self.ceo = ceo

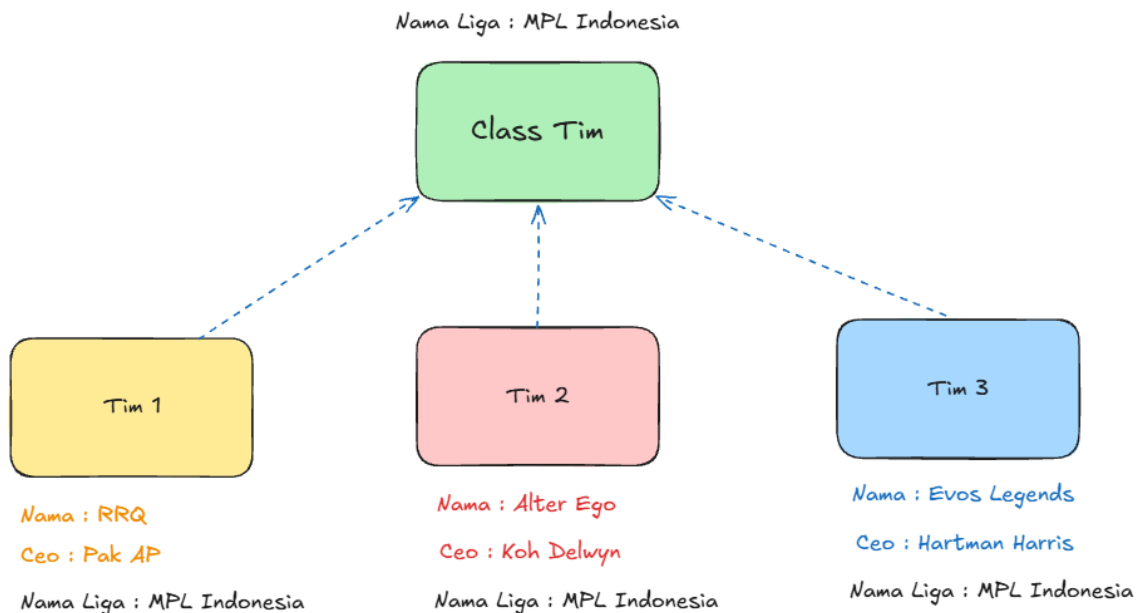
rrq = Tim("RRQ", "Pak AP")
evos = Tim("Evos Legends", "Hartman Harris")

print(rrq.nama_liga)    # MPL Indonesia
print(evos.nama_liga)   # MPL Indonesia
```

Cara mengubah atribut kelas dengan benar adalah wajib lewat nama kelasnya supaya perubahan itu berlaku untuk semua objek sekaligus.

```
Tim.nama_liga = "MPL Indonesia Season 15"

print(rrq.nama_liga)    # MPL Indonesia Season 15
print(evos.nama_liga)   # MPL Indonesia Season 15
```





Kita dapat lihat bahwa nama dan ceo adalah atribut instance dimana masing masing objek memiliki atribut nama dan ceo yang berbeda. Lalu ada atribut kelas yang menjadi default dari nama liga untuk masing masing tim yaitu MPL Indonesia dan ketika atribut kelasnya diubah maka akan mempengaruhi atribut dari objek yang berasal dari kelas tersebut yaitu Nama Liga. Berikut adalah gabungan kode penerapan antara atribut instance dan kelas.

```
class Tim:
    nama_liga = "MPL Indonesia"

    def __init__(self, nama, ceo):
        self.nama = nama
        self.ceo = ceo

rrq      = Tim("RRQ Hoshi", "Pak AP")
evos     = Tim("Evos Legends", "Hartman Harris")
alter_ego = Tim("Alter Ego", "Koh Delwyn")

print(f"{rrq.nama} (CEO: {rrq.ceo}) - Liga: {Tim.nama_liga}\n")
print(f"{evos.nama} (CEO: {evos.ceo}) - Liga: {Tim.nama_liga}\n")
print(f"{alter_ego.nama} (CEO: {alter_ego.ceo}) - Liga: {Tim.nama_liga}\n")

rrq.ceo = "Pak AP Manullang"
print(f"{rrq.nama} (CEO: {rrq.ceo}) - Liga: {Tim.nama_liga}\n")

Tim.nama_liga = "MPL Indonesia Season 15"
print(f"{rrq.nama} (CEO: {rrq.ceo}) - Liga: {Tim.nama_liga}\n")
print(f"{evos.nama} (CEO: {evos.ceo}) - Liga: {Tim.nama_liga}\n")
print(f"{alter_ego.nama} (CEO: {alter_ego.ceo}) - Liga: {Tim.nama_liga}\n")
```



## Method

Selain atribut, ada method yaitu sebuah fungsi yang berada di dalam class dibagi menjadi tiga jenis yaitu **instance method**, **class method**, dan **static method**. Ketiganya terlihat mirip karena sama-sama ditulis di dalam badan class tetapi memiliki cara kerja, tujuan, dan cara pemanggilan yang cukup berbeda.

| Instance Method                         | Class Method                       | Static Method                                 |
|-----------------------------------------|------------------------------------|-----------------------------------------------|
| Tidak Memiliki Decorator                | @classmethod                       | @staticmethod                                 |
| self                                    | cls                                | Tidak memiliki parameter utama                |
| Dipanggil via objek                     | Dipanggil via class                | Dipanggil via class                           |
| Bisa mengakses atribut instance & kelas | Hanya bisa mengakses atribut kelas | Tidak bisa mengakses atribut instance & kelas |

### Instance Method

Instance method adalah jenis method yang paling sering digunakan karena method ini yang bekerja langsung dengan data milik satu objek tertentu. Parameter pertamanya selalu self yang merujuk pada objek yang memanggil method tersebut. Karena punya akses ke self, instance method bisa membaca maupun mengubah atribut instance milik objek dan juga tetap bisa membaca atribut kelas.

```
class Pertandingan:
    def __init__(self, tim_a, tim_b):
        self.tim_a = tim_a
        self.tim_b = tim_b
        self.skor_a = 0
        self.skor_b = 0
        self.selesai = False
```



```
def tambah_skor(self, tim, poin=1):
    if tim == self.tim_a:
        self.skor_a += poin
    elif tim == self.tim_b:
        self.skor_b += poin
    else:
        print(f"{tim} tidak terdaftar di pertandingan ini!")

def selesaikan(self):
    self.selesai = True

def tampilkan_hasil(self):
    status = "Selesai" if self.selesai else "Berlangsung"
    print(f"{self.tim_a} {self.skor_a} - {self.skor_b} {self.tim_b} ({status})")

final = Pertandingan("RRQ", "Evos Legends")
final.tambah_skor("RRQ", 2)
final.tambah_skor("Evos Legends", 1)
final.selesaikan()
final.tampilkan_hasil()  # RRQ 2 - 1 Evos Legends (Selesai)
```

## Class Method

Class method adalah method yang menerima cls sebagai parameter pertama, parameter cls ini merujuk ke class itu sendiri sehingga class method hanya bisa mengakses atribut kelas dan tidak tahu-menahu soal data milik satu objek tertentu.

Salah satu kegunaan class method yang paling umum adalah sebagai factory method, yaitu cara alternatif untuk membuat objek, selain lewat konstruktor `__init__` biasa. Ini berguna misalnya ketika data dalam format yang berbeda-beda (dictionary, baris teks, hasil parsing dari sumber lain) dan kita ingin cara pembuatan objek yang lebih praktis untuk tiap format tersebut.



```
class Jadwal:
    musim_liga = "MPL Indonesia Season 14"

    def __init__(self, tim_a, tim_b, tanggal):
        self.tim_a = tim_a
        self.tim_b = tim_b
        self.tanggal = tanggal

    # Class method sebagai factory method -- buat objek dari dictionary
    def dari_dict(cls, data):
        """Alternatif konstruktor: buat Jadwal dari data berbentuk
        dictionary."""
        return cls(data["tim_a"], data["tim_b"], data["tanggal"])

    @classmethod
    def ganti_musim(cls, musim_baru):
        """Mengubah musim liga aktif, berlaku untuk seluruh objek
        Jadwal."""
        cls.musim_liga = musim_baru

    def info(self):
        print(f"{self.tim_a} vs {self.tim_b} - {self.tanggal}
        ({Jadwal.musim_liga})\n")

data_laga = {"tim_a": "RRQ", "tim_b": "Evos Legends", "tanggal": "12
September 2026"}
data_laga2 = {"tim_a": "ONIC", "tim_b": "Alter Ego", "tanggal": "13
September 2026"}
laga1 = Jadwal.dari_dict(data_laga)
laga2 = Jadwal.dari_dict(data_laga2)
laga1.info()    # RRQ vs Evos Legends - 12 September 2026 (MPL Indonesia
Season 14)
laga2.info()    # ONIC vs Alter Ego - 13 September 2026 (MPL Indonesia
Season 14)

Jadwal.ganti_musim("MPL Indonesia Season 15")
laga1.info()    # RRQ vs Evos Legends - 12 September 2026 (MPL Indonesia
Season 15)
laga2.info()    # ONIC vs Alter Ego - 13 September 2026 (MPL Indonesia
Season 15)
```



## Static Method

Static method adalah jenis method yang dimana tidak menerima self maupun cls sehingga tidak terikat dengan data objek tertentu ataupun data kelas. Static method pada dasarnya hanyalah fungsi biasa yang dititipkan di dalam sebuah class, karena secara konsep memang berhubungan erat dengan class tersebut meskipun secara teknis tidak membutuhkan data apa pun dari class atau objeknya.

Static method biasanya dipakai untuk fungsi bantu yang masih berhubungan dengan class tersebut, tapi tidak butuh data dari objek maupun dari class-nya. Contohnya fungsi untuk mengecek apakah suatu input valid, mengubah satu jenis data ke jenis lain, atau menghitung sesuatu yang hasilnya cukup didapat dari input yang diberikan saja. Dengan menaruh fungsi-fungsi seperti ini sebagai static method, kode jadi lebih rapi karena fungsi yang sejenis dikumpulkan dalam satu class.

```
class UtilitasPertandingan:

    @staticmethod
    def validasi_nama_tim(nama_tim):
        tim_terdaftar = ["RRQ", "Evos Legends", "Alter Ego", "ONIC"]
        return nama_tim in tim_terdaftar

    @staticmethod
    def tentukan_pemenang(tim_a, skor_a, tim_b, skor_b):
        if skor_a > skor_b:
            return tim_a
        elif skor_b > skor_a:
            return tim_b
        return "Seri"

print(UtilitasPertandingan.validasi_nama_tim("RRQ"))           # True
print(UtilitasPertandingan.validasi_nama_tim("Team Baru"))     # False
print(UtilitasPertandingan.tentukan_pemenang("RRQ", 2, "Evos Legends", 1))
# RRQ
```



.Ini merupakan contoh penerapan kode secara keseluruhannya.

```
class Pertandingan:
    musim_liga = "MPL Indonesia Season 14"

    def __init__(self, tim_a, tim_b):
        self.tim_a = tim_a
        self.tim_b = tim_b
        self.skor_a = 0
        self.skor_b = 0
        self.selesai = False

    def tambah_skor(self, tim, poin=1):
        if tim == self.tim_a:
            self.skor_a += poin
        elif tim == self.tim_b:
            self.skor_b += poin
        else:
            print(f"{tim} tidak terdaftar di pertandingan ini!")

    def selesaikan(self):
        self.selesai = True

    def tampilkan_hasil(self):
        status = "Selesai" if self.selesai else "Berlangsung"
        print(f"{self.tim_a} {self.skor_a} - {self.skor_b} {self.tim_b} "
              f"({status}, {Pertandingan.musim_liga})")

    @classmethod
    def dari_dict(cls, data):
        return cls(data["tim_a"], data["tim_b"])

    @classmethod
    def ganti_musim(cls, musim_baru):
        cls.musim_liga = musim_baru

    @staticmethod
    def validasi_nama_tim(nama_tim):
        tim_terdaftar = ["RRQ", "Evos Legends", "Alter Ego", "ONIC"]
        return nama_tim in tim_terdaftar
```



```
@staticmethod
def tentukan_pemenang(tim_a, skor_a, tim_b, skor_b):
    if skor_a > skor_b:
        return tim_a
    elif skor_b > skor_a:
        return tim_b
    return "Seri"

print(Pertandingan.validasi_nama_tim("RRQ"))
print(Pertandingan.validasi_nama_tim("Team Baru"))

final = Pertandingan("RRQ", "Evos Legends")
final.tambah_skor("RRQ", 2)
final.tambah_skor("Evos Legends", 1)
final.selesaikan()
final.tampilkan_hasil()

data_laga = {"tim_a": "ONIC", "tim_b": "Alter Ego"}
laga2 = Pertandingan.dari_dict(data_laga)
laga2.tambah_skor("ONIC", 2)
laga2.tampilkan_hasil()

Pertandingan.ganti_musim("MPL Indonesia Season 15")
final.tampilkan_hasil()
laga2.tampilkan_hasil() pemenang = Pertandingan.tentukan_pemenang("RRQ",
final.skor_a, "Evos Legends", final.skor_b)
print(f"Pemenang: {pemenang}")
```